

操作系统第三次实验报告

李汶峰

202300460158

2023 级网安一班

2025 年 4 月 24 日

1 实验目的

- 学习信号量机制：通过实现信号量操作（创建、等待、释放），深入理解信号量在解决资源竞争和任务同步中的作用。
- 理解生产者-消费者问题、哲学家就餐问题、读者-写者问题中作为操作系统中经典并发控制模型的核心思想，理解多任务协作中的同步与互斥机制。
- 熟悉 $\mu\text{C}/\text{OS-II}$ 实时操作系统的使用

2 任务一

2.1 问题描述

生产者-消费者问题是操作系统中研究多任务同步的经典问题。生产者生成数据并存入有限大小的缓冲区，消费者从缓冲区取出数据处理，需通过同步机制避免缓冲区溢出或空读，并通过互斥机制防止并发访问冲突。本实验基于 $\mu\text{C}/\text{OS-II}$ 实时操作系统，使用信号量实现生产者-消费者模型，模拟生产与消费过程。

2.2 实验过程

一、新增变量的定义

$\text{buffer}[N]$ 表示共享缓冲区， in , out 表示缓冲区的生产和消费位置指针， count 为当前缓冲区中的商品数量， produced_item , consumed_item 表示用于跟踪生产和消费的商品编号。

```
1 int buffer[N];
2 int in = 0;
3 int out = 0;
4 int count = 0;
5 int produced_item = 0;
6 int consumed_item = 0;
```

二、生产者代码的实现

1. 生产者等待缓冲区（表示空闲位置数），即 `OSSemPend(empty)`，当缓冲区不为空时进行生产工作。
2. 生产者获取互斥锁 `OSSemPend(mutex)`，对接下来的生产操作上锁以防止其他进程进入。
3. 生产者开始放入商品，通过打印 `Producer` 来表示生产过程。
4. 生产完成需要释放互斥锁，即 `OSSemPost(mutex)`。
5. 增加满缓冲区的计数，表示将生产好的商品放入满缓冲区中，即 `OSSemPost(full)`。
6. 最后，通过 `OSTimeDly(200)` 让生产者任务挂起一段时间，以让出 CPU。

```
1 static void producer(void *p_arg)
2 {
3     INT8U err;
4     int item = 0;
5
6     for (;;)
7     {
8
9         item++;
10        print_str("Producer: trying to produce item ");
11        char buf[20];
12        sprintf(buf, "%d\n", item);
13        print_str(buf);
14
15        OSSemPend(empty, 0, &err); // 等待空缓冲区
16        OSSemPend(mutex, 0, &err);
17
18
19        buffer[in] = item;
20        in = (in + 1) % N;
21        count++;
22
23        print_str("Producer: put item ");
24        sprintf(buf, "%d", item);
25        print_str(buf);
26        print_str(" into buffer. Buffer count: ");
27        sprintf(buf, "%d\n", count);
28        print_str(buf);
29
30        OSSemPost(mutex);
31        OSSemPost(full); // 增加满缓冲区计数
```

```
32
33     OSTimeDly(200); // 延迟一段时间
34 }
35 }
```

二、消费者代码的实现

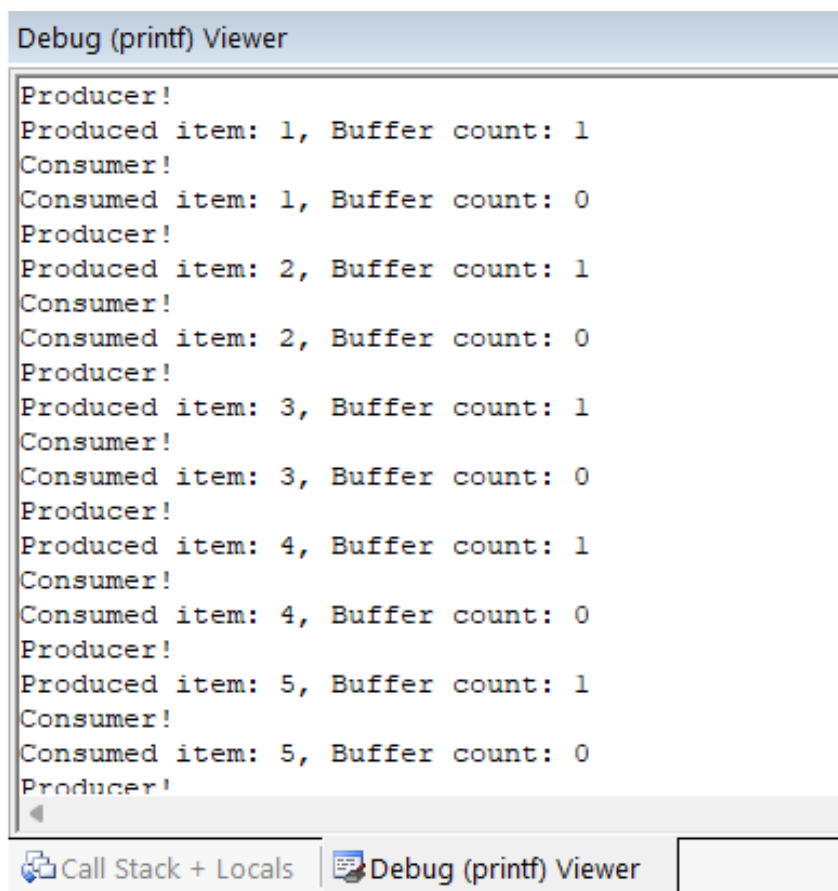
1. 消费者等待满缓冲区，当满缓冲区有商品时开始消费操作，即 OSSemPend(full).
2. 接着消费者获取互斥锁 OSSemPend(mutex) 确保同一时间只有一个任务能访问缓冲区。
3. 消费者实行消费操作，从满缓冲区中取出商品并打印 Consumer 表示消费过程。
4. 释放互斥锁 OSSemPost(mutex)
5. 消费完成后增加空缓冲区的计数 OSSemPost(empty)，以通知生产者。
6. 最后 OSTimeDly(300) 让出 CPU，给生产者任务运行

```
1 static void consumer(void *p_arg)
2 {
3     INT8U err;
4     int item;
5
6     for (;;)
7     {
8         print_str("Consumer: trying to consume item\n");
9
10        OSSemPend(full, 0, &err); // 等待满缓冲区
11        OSSemPend(mutex, 0, &err);
12
13        // 从缓冲区取出
14        item = buffer[out];
15        out = (out + 1) % N;
16        count--;
17
18        print_str("Consumer: got item ");
19        char buf[20];
20        sprintf(buf, "%d", item);
21        print_str(buf);
22        print_str(" from buffer. Buffer count: ");
23        sprintf(buf, "%d\n", count);
24        print_str(buf);
25
26        OSSemPost(mutex);
27        OSSemPost(empty); // 增加空缓冲区计数
```

```
28  
29     OSTimeDly(300); // 延迟一段时间  
30 }  
31 }
```

2.3 实验结果

实验结果如图 1 所示，生产者首先生成一个商品，该商品编号为 1；接着消费者进行消费，消费的商品编号也为 1，后面生产者和消费者轮流进行工作。生产者要在缓冲区不为满时才生产，消费者需要在缓冲区不空时才能进行消费，实现了生产者-消费者问题。



```
Debug (printf) Viewer  
Producer!  
Produced item: 1, Buffer count: 1  
Consumer!  
Consumed item: 1, Buffer count: 0  
Producer!  
Produced item: 2, Buffer count: 1  
Consumer!  
Consumed item: 2, Buffer count: 0  
Producer!  
Produced item: 3, Buffer count: 1  
Consumer!  
Consumed item: 3, Buffer count: 0  
Producer!  
Produced item: 4, Buffer count: 1  
Consumer!  
Consumed item: 4, Buffer count: 0  
Producer!  
Produced item: 5, Buffer count: 1  
Consumer!  
Consumed item: 5, Buffer count: 0  
Producer!
```

图 1: 任务一实验结果

3 任务二

3.1 问题描述

想象五位哲学家围坐在一张圆桌上，每人面前有一盘食物，但他们需要用两把叉子（或筷子）才能进餐。桌子上共有五把叉子，每把叉子位于两位哲学家之间，供相邻的两位哲学家共享。哲学家的行为模式是：思考、饥饿、进餐，然后重复。每位哲学家交替进行思考（不需要叉子）和进餐（需要两把叉子）。进餐时，哲学家必须同时持有左右两把叉子。叉子是共享资源，每次只能被一位哲学家使用。

3.2 实验过程

一、关键实验说明

1. 进程切换策略：

- 使用 OSTimeDly 实现任务切换，每个哲学家在不同阶段有随机延迟
- 预定义的 delay_times 数组为每个哲学家设置不同的基础延迟时间

2. 死锁避免策略：

- 采用”部分顺序法”：偶数编号哲学家先拿左筷子，奇数编号先拿右筷子

3. 饥饿问题解决方式：

- 高优先级哲学家（低编号）在获取筷子前有额外等待
- 高优先级哲学家就餐后有更长冷却时间

4. 优先级设计：

- 哲学家任务优先级从 1 到 5(1 最高)
- 高优先级哲学家通过额外等待平衡资源获取机会

二、实验代码分析

定义了哲学家的数量为 5 位及哲学家状态的定义，以及筷子信号量数组、哲学家状态数组、延迟时间数组和函数声明。

```
1 #define PHILOSOPHER_NUM 5
2 #define THINKING 0
3 #define HUNGRY 1
4 #define EATING 2
5
6 static OS_STK philosopher_stk[PHILOSOPHER_NUM][TASK_STK_SIZE];
7 OS_EVENT *chopsticks[PHILOSOPHER_NUM];
8 int philosopher_state[PHILOSOPHER_NUM];
9 int delay_times[PHILOSOPHER_NUM] = {200, 150, 300, 250, 350};
10
11 void philosopher(void *p_arg);
12 void print_str(char *);
13 void print_philosopher_state(int id, int state);
```

下面这段 main 函数创建了五根筷子的信号量、五个哲学家进程任务，最后启动调度器开始运行。

```
1 int main(void)
2 {
3     int i;
4     OSInit();
5 }
```

```

6 // Initialize hardware and semaphores
7 systick_init();
8 for(i = 0; i < PHILOSOPHER_NUM; i++) {
9     chopsticks[i] = OSSemCreate(1);
10    philosopher_state[i] = THINKING;
11 }
12
13 // Create philosopher tasks with different priorities
14 for(i = 0; i < PHILOSOPHER_NUM; i++) {
15     OSTaskCreate(philosopher,
16                 (void *)i,
17                 &philosopher_stk[i][TASK_STK_SIZE-1],
18                 i+1);
19 }
20
21 OSStart();
22 return 0;
23 }

```

philosopher 函数实现了哲学家就餐问题中单个哲学家的完整行为逻辑：每位哲学家在循环中交替进行思考、饥饿等待、获取筷子、就餐和释放筷子的过程。通过奇偶编号区分筷子获取顺序（偶数编号先左后右，奇数编号先右后左）来预防死锁，同时为高优先级哲学家添加额外等待时间防止饥饿。每个阶段通过状态变量跟踪并打印详细状态信息，且思考/就餐时间采用基础延迟加随机扰动的方式模拟真实行为，最后严格遵循获取-释放的信号量操作确保资源共享安全。

```

1 static void philosopher(void *p_arg)
2 {
3     INT8U err;
4     int id = (int)p_arg;
5     int left = id;
6     int right = (id + 1) % PHILOSOPHER_NUM;
7
8     for (;;) {
9         // Thinking phase
10        philosopher_state[id] = THINKING;
11        print_philosopher_state(id, THINKING);
12        OSTimeDly(delay_times[id] + (rand() % 100));
13
14        // Hungry phase
15        philosopher_state[id] = HUNGRY;
16        print_philosopher_state(id, HUNGRY);
17
18        // Acquire chopsticks (with deadlock prevention)
19        if(id % 2 == 0) {

```

```
20         OSSemPend(chopsticks[left], 0, &err);
21         OSSemPend(chopsticks[right], 0, &err);
22     } else {
23         if(id < 2) OSTimeDly(50); // Anti-starvation delay
24         OSSemPend(chopsticks[right], 0, &err);
25         OSSemPend(chopsticks[left], 0, &err);
26     }
27
28     // Eating phase
29     philosopher_state[id] = EATING;
30     print_philosopher_state(id, EATING);
31     OSTimeDly(delay_times[id] + (rand() % 100));
32
33     // Release chopsticks
34     OSSemPost(chopsticks[left]);
35     OSSemPost(chopsticks[right]);
36
37     print_str("Philosopher ");
38     char msg[20];
39     sprintf(msg, "%d", id);
40     print_str(msg);
41     print_str(" has finished eating \n");
42
43     if(id < 2) OSTimeDly(100); // Anti-starvation delay
44 }
45 }
```

3.3 实验结果

```
Debug (printf) Viewer
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 1 is hungry and waiting for chopsticks
Philosopher 1 is now eating
Philosopher 0 is hungry and waiting for chopsticks
Philosopher 2 is hungry and waiting for chopsticks
Philosopher 3 is hungry and waiting for chopsticks
Philosopher 3 is now eating
Philosopher 0 is now eating
Philosopher 1 has finished eating
Philosopher 4 is hungry and waiting for chopsticks
Philosopher 1 is thinking
Philosopher 0 has finished eating
Philosopher 2 is now eating
Philosopher 3 has finished eating
Philosopher 3 is thinking
Philosopher 4 is now eating
Philosopher 1 is hungry and waiting for chopsticks
Philosopher 0 is thinking
Philosopher 3 is hungry and waiting for chopsticks
Philosopher 0 is hungry and waiting for chopsticks
```

图 2: 任务二实验结果

4 任务三

4.1 问题描述

读者-写者问题模拟了多个进程或线程对共享资源（如数据库或文件）的访问，分为两类角色：

- 读者（Reader）：只读取共享资源的数据，不修改内容。多个读者可以同时访问资源，因为读取操作不影响数据一致性。
- 写者（Writer）：修改共享资源的内容。写者访问时需要独占资源，防止其他读者或写者在同一时间访问，以确保数据一致性。

目标：设计一种同步机制，确保：

- 多个读者可以同时读取共享资源（并发性）。
- 写者访问时具有互斥性（独占资源）。
- 避免死锁、饥饿（如写者长期无法写入或读者长期无法读取）。
- 提高系统效率，最大化并发访问。

4.2 实验过程

1. **写者代码实现**：写者任务在独占 `rw_mutex` 锁才能修改共享数据，每次写入会打印数据变更前后的值，通过低优先级（4-5）和随机延迟避免饥饿问题，严格的互斥机制保证数据一致性。

```

1 static void writer(void *p_arg)
2 {
3     INT8U err;
4     int id = (int)p_arg;
5     char msg[50];
6
7     for (;;)
8     {
9         // 写者进入区
10        OSSemPend(rw_mutex, 0, &err);
11
12        // 写操作
13        sprintf(msg, "Writer %d writes: %d->%d\n", id, shared_data, ←
        ↪shared_data+1);
14        print_str(msg);
15        shared_data++; // 修改共享数据
16        OSTimeDly(30 + (rand() % 30)); // 随机写入时间
17
18        // 写者退出
19        OSSemPost(rw_mutex);
20
21        OSTimeDly(150 + (rand() % 100)); // 随机思考时间
22    }
23 }

```

2. 读者代码实现:读者任务通过 mutex 信号量保护 read_count 计数器,首个读者获取 rw_mutex 读写锁后,允许多读者并发读取共享数据,最后一位读者释放锁;采用随机延迟模拟读取耗时,并通过高优先级 (1-3) 实现**读者优先策略**,确保读操作的高吞吐量。

```

1 static void reader(void *p_arg)
2 {
3     INT8U err;
4     int id = (int)p_arg;
5     char msg[50];
6
7     for (;;)
8     {
9         // 读者进入
10        OSSemPend(mutex, 0, &err);
11        if(++read_count == 1) {
12            OSSemPend(rw_mutex, 0, &err);
13        }
14        OSSemPost(mutex);

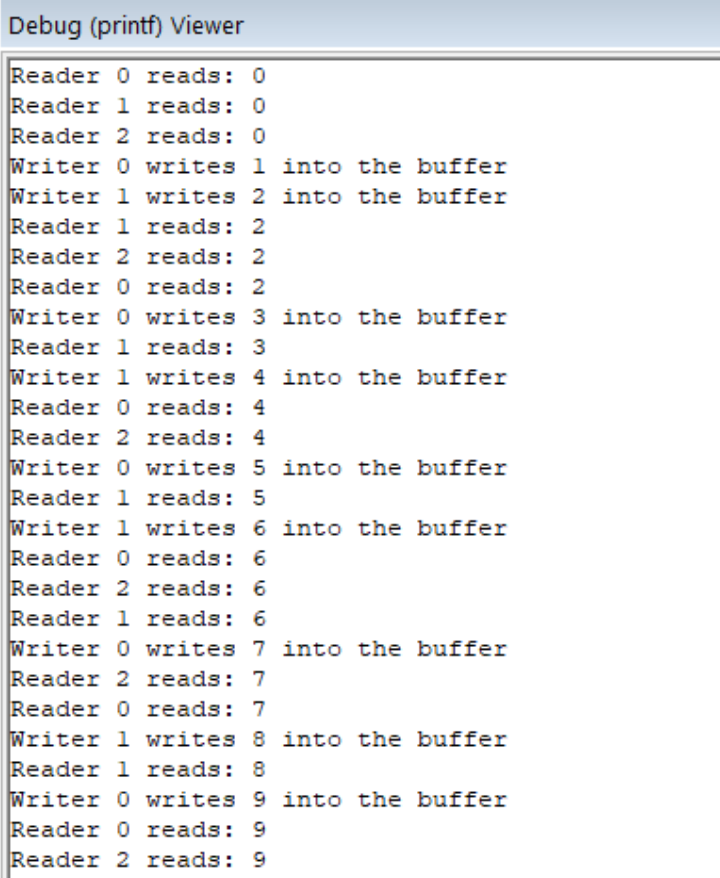
```

```

15
16     // 读操作
17     sprintf(msg, "Reader %d reads: %d\n", id, shared_data);
18     print_str(msg);
19     OSTimeDly(50 + (rand() % 50)); // 随机读取时间
20
21     // 读者退出区
22     OSSemPend(mutex, 0, &err);
23     if(--read_count == 0) {
24         OSSemPost(rw_mutex);
25     }
26     OSSemPost(mutex);
27
28     OSTimeDly(100 + (rand() % 100)); // 随机思考时间
29 }
30 }

```

4.3 实验结果



```

Debug (printf) Viewer
Reader 0 reads: 0
Reader 1 reads: 0
Reader 2 reads: 0
Writer 0 writes 1 into the buffer
Writer 1 writes 2 into the buffer
Reader 1 reads: 2
Reader 2 reads: 2
Reader 0 reads: 2
Writer 0 writes 3 into the buffer
Reader 1 reads: 3
Writer 1 writes 4 into the buffer
Reader 0 reads: 4
Reader 2 reads: 4
Writer 0 writes 5 into the buffer
Reader 1 reads: 5
Writer 1 writes 6 into the buffer
Reader 0 reads: 6
Reader 2 reads: 6
Reader 1 reads: 6
Writer 0 writes 7 into the buffer
Reader 2 reads: 7
Reader 0 reads: 7
Writer 1 writes 8 into the buffer
Reader 1 reads: 8
Writer 0 writes 9 into the buffer
Reader 0 reads: 9
Reader 2 reads: 9

```

图 3: 任务三实验结果

5 实验感悟

本次操作系统实验通过实现生产者-消费者问题、哲学家就餐问题和读者-写者问题，掌握了 $\mu\text{C}/\text{OS-II}$ 实时操作系统的信号量机制和任务管理，也详细了解这三个任务的工作原理。

三个问题都是操作系统并发控制的经典模型，理论上分别聚焦不同的资源竞争场景：生产者-消费者关注缓冲区同步，读者-写者强调多读者并发与写者互斥，哲学家就餐突出死锁与资源循环等待。通过实验写出 $\mu\text{C}/\text{OS-II}$ 的实际程序，深刻体会到理论模型在真实系统中的复杂性。

实验中广泛使用了 $\mu\text{C}/\text{OS-II}$ 的信号量函数（OSSemCreate、OSSemPend、OSSemPost），让我对信号量的创建、等待和释放有了全面认识。特别是 OSSemPend（P 操作）和 OSSemPost（V 操作）两个函数，让我看到信号量如何在底层支持任务同步。